CS 3960 (3)

Vibe Coding

Spring 2026, *content* Pavel Panchekha and John Regehr, *illustrations* Gemini 3 Nano Banana Pro

- Reading assignment due Friday
  - "Why fuzzing still matters in memory-safe languages"
- How's the code coverage assignment going?
- Let's talk about your next assignment — in-class Demo Day next Weds

Fuzzing and **random testing** mean the same thing:

Generating test cases randomly.

However, this is a huge space with a lot of variation in how things are done.

What kind of software should be fuzzed?

Everything!

But especially security boundaries.

Random testing is how most security-related bugs are found.

What are some security boundaries?

There's research showing that LLMs are not good at writing secure code.

And neither are people, generally.

Regardless of AI support, be incredibly careful any time you're writing code at a security boundary.

The simplest fuzzers just make up totally random data.

In like the 90s this was good enough to crash most any software.

Can you think of any problems with totally random tests?

Some fuzzers **generate** each input from scratch

Other fuzzers **mutate** inputs

Strengths and weaknesses of each technique?

Fuzzing problem #1: Validity

What happens if you fuzz a Rust compiler using random bytes?

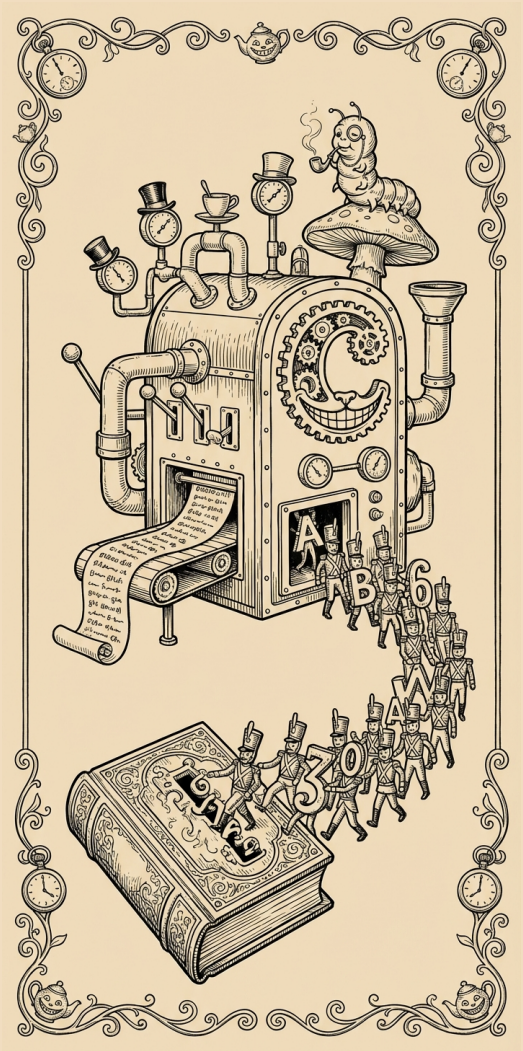
Solutions to the validity problem:

1. Fixup invalid inputs
2. Only generate valid inputs in the first place
3. Mutate valid inputs in a validity-preserving way
4. Infer validity by coverage

Fixing up invalid inputs

Often, a checksum

This is a great technique, but has niche applicability



Only generate valid inputs

This can be a lot of work!

My group spent several years creating Csmith, which generates valid C code

LLMs are changing this!

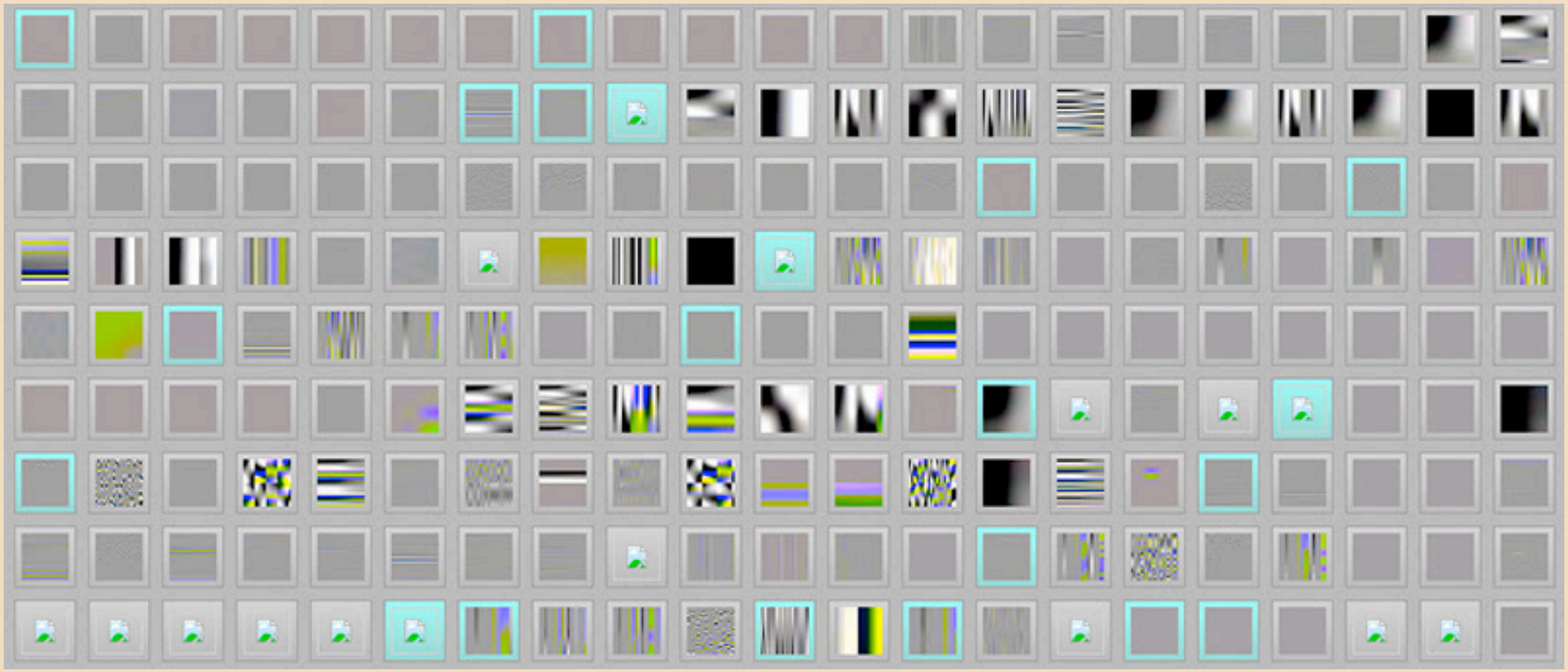
Preserving validity while mutating inputs can also be a lot of work

But, again, LLMs are changing this

Infer validity by coverage

1. Instrument the system under test, so it reports coverage
2. Assume that test cases are interesting if they cover a new part of the system
3. Discard uninteresting test inputs, and mutate interesting ones

Example: Coverage-driven fuzzing of a JPEG decoder using AFL



Around 10 years ago AFL totally changed the game for fuzzing

AFL Demo!

Fuzzing Problem #2: Expressiveness

An expressive fuzzer explores every part of the input space with sufficiently high probability

Achieving expressiveness is very difficult in practice.

How can we tell if our fuzzer is insufficiently expressive?

How can we tell if our fuzzer is sufficiently expressive?

Fuzzing Problem #3: The Oracle Problem

How can we tell whether some particular test case triggered a bug in our system?

We've already looked at oracles—does random testing make the oracle problem easier or harder?

A few additional oracles for fuzzing:

- Function-inverse pairs
- Metamorphic testing

There's a lot of scope for creativity here

Practical advice for fuzzing:

Do it.

Start early. If you have a simple system, write a simple fuzzer for it. Then evolve both the system and the fuzzer.

If you don't find bugs, your customers will.

If you and your customers don't find bugs, your adversaries will.

More reading:

How to Fuzz an ADT Implementation

The Fuzzing Book

- In the "syllabus" repo in the class GitHub org, find this file: `slides/a5-fpmul.c`
- This is a simple floating point multiplier
- It's wrong, it gives an incorrect answer for ~0.000001% of inputs
- Write a fuzzer in C or C++ that finds a pair of inputs where it is wrong
  - Print them using the `%a` printf specifier
  - Enter these into the quiz response
  - Only test "normal" FP values: make sure that both inputs and also the expected output are not NaNs, infs, or denormals
  - Have one of us look at your answers if you're unsure!